

Assessment II

Vorname: _____

Punkte: ____ / 90, Note: ____

Name: _____

Frei lassen für Korrektur.

Klasse: 4ibb1

Hilfsmittel:

- Notizen, ein A4-Blatt, Font `Courier New`, ≥ 10 pt.
- Löse die Aufgaben jeweils direkt auf den Prüfungsblättern.
- Zusatzblätter, falls nötig, mit Namen und Frage-Nr. auf jedem Blatt.

Nicht erlaubt:

- Unterlagen (Slides, Bücher, ...).
- Computer (Laptop, Smartphone, ...).
- Kommunikation (mit Personen, KI, ...).

Bewertung:

- Multiple Response: ☐ *Ja* oder ☐ *Nein* ankreuzen, +1/-1 Punkt pro richtige/falsche Antwort, beide nicht ankreuzen ergibt +0 Punkte; Total pro Frage gibt es nie weniger als 0 Punkte.
- Offene Fragen: Bewertet wird Korrektheit, Vollständigkeit und Kürze der Antwort.
- Programme: Bewertet wird die Idee/Skizze und Umsetzung des Programms.

Fragen zur Prüfung:

- Während der Prüfung werden vom Dozent keine Fragen zur Prüfung beantwortet.
- Ist etwas unklar, mache eine Annahme und notiere diese auf der Prüfung.

Threads und Synchronisation

1) Schreibe ein Programm *sum*, das *n* per Command Line übergebene Zahlen parallel, in *n* Threads, zu *int* konvertiert und aufaddiert, dann in *main()* die Summe ausgibt. Punkte: _ / 16

```
$ ./sum 3 4 2
9
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int atoi(const char *nptr); // convert a string to an integer
int printf(const char *format, ...); // format string %s, char %c, int %d

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start) (void *), void *arg); // starts a thread; attr = NULL
int pthread_join(pthread_t thread, void **retval); // retval = NULL

int pthread_mutex_lock(pthread_mutex_t *mutex); // lock a mutex
int pthread_mutex_unlock(pthread_mutex_t *mutex); // unlock a mutex
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // initialize a mutex
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

IPC mit Pipes

2) Welche der folgenden Aussagen über FIFOs (Named Pipes) sind korrekt? Punkte: _ / 4

Zutreffendes ankreuzen

- ☐ Ja | ☐ Nein Zwei Prozesse können mehr als ein FIFO Objekt parallel nutzen.
- ☐ Ja | ☐ Nein Bei FIFOs geht es um den Message-basierten Transfer von Daten.
- ☐ Ja | ☐ Nein Ein FIFO Objekt erlaubt den Datenaustausch in beide Richtungen.
- ☐ Ja | ☐ Nein Bei *write()* blockiert ein FIFO Objekt, bis die Gegenseite bereit ist.

|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|
|

Weitere Aufgabe auf der nächsten Seite ...

Sockets

3) Schreibe ein Programm, das UNIX Domain Byte-Stream Requests auf dem Pfad `"/log"`

empfängt, max. 1024 Bytes, und jeden Request auf `STDOUT_FILENO` ausgibt. Punkte: / 16

Verwende die folgenden Calls (soweit sinnvoll), ohne `#includes` und Fehlerbehandlung:

```
int accept(int socket, struct sockaddr *addr, socklen_t *addrlen); //  
accept a connection on a socket; returns the accepted client socket  
int bind(int socket, struct sockaddr *addr, socklen_t addrlen); // bind  
int listen(int socket, int backlog); // listen for connections on socket  
int socket(int domain, int type, int protocol); // create an endpoint for  
communication; domain = AF_INET, AF_UNIX; type = SOCK_STREAM, SOCK_DGRAM;  
protocol = 0; returns a file descriptor for the new socket or -1 on error
```

```
struct sockaddr_un { // UNIX domain socket address structure  
    sa_family_t sun_family; // AF_UNIX  
    char sun_path[108]; // Pathname  
};
```

```
int close(int fd); // close a file descriptor  
ssize_t read(int fd, void *buf, size_t count); // read from a file descr.  
ssize_t write(int fd, const void *buf, size_t count); // write to a file
```

```
char *strcpy(char *dest, char *src); // copy a string src to buffer dest
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

POSIX IPC

4) Schreibe ein Programm *randmq*, das die Buchstaben 'a' bis 'z' über eine bestehende POSIX Message Queue *"/mq"* an seinen Child-Prozess sendet, mit je einer zufälligen Priorität von 0 bis 9, und auf Empfangsseite beides auf die Konsole ausgibt, wie hier gezeigt. Punkte: / 18

Hinweis: mq_receive() erwartet Bufferlänge >= attr.mq_msgsize, siehe auch mq_getattr().

```
$ ./randmq
i (7)
e (3)
...
```

Verwende die folgenden Calls (soweit sinnvoll), ohne #includes und Fehlerbehandlung:

```
pid_t fork(void); // create a child process, returns 0 in child process
void exit(int status); // cause normal process termination
```

```
mqd_t mq_open(char *name, int flags); // open a message queue; returns a
message queue descriptor; flags include O_RDONLY, O_WRONLY and O_RDWR;
```

```
struct mq_attr { long mq_maxmsg, long mq_msgsize, ... }; // mq attributes
int mq_getattr(mqd_t mqd, struct mq_attr *attr); // get mq attributes;
returns 0 on success
int mq_send(mqd_t mqd, char *msg, size_t len, unsigned int prio);
// send a message to a message queue; returns 0 on success
ssize_t mq_receive(mqd_t mqd, char *msg, size_t len, unsigned int *prio);
// receive a message from a mq; blocking; returns # of bytes in message
```

```
int printf(const char *format, ...); // format string %s, char %c, int %d
long random(void); // returns a random number, use value with modulo %
```

|

|

|

|

|

Idee (kurz) und C Source Code auf Folgeseite, oder auf Zusatzblatt mit Namen, Frage-Nr.

(4) Fortsetzung:

|

|

|

|

|

|

|

|

|

Weitere Aufgabe auf der nächsten Seite ...

5) Schreibe ein Programm, welches 5 Enten simuliert, die gleichzeitig versuchen, eins von 3 Stücken Brot zu ergattern. Nutze Threads und Semaphore so, dass exakt 3 Enten je ein Stück Brot bekommen. Diese geben "gulp" aus, alle anderen "quack".

Punkte: _ / 14

Verwende die folgenden Calls (soweit sinnvoll), ohne #includes und Fehlerbehandlung:

```
int printf(const char *format, ...); // format string %s, char %c, int %d

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start) (void *), void *arg); // starts a thread; attr = NULL
int pthread_detach(pthread_t thread); // detach a thread
void pthread_exit(void *retval); // terminate calling thread, no return

int sem_init(sem_t *sem, int pshared, unsigned int value); // initialize
an unnamed semaphore, pshared = 0
int sem_wait(sem_t *s); // decrement a semaphore, blocking if <= 0
int sem_trywait(sem_t *sem); // returns 0 on success, -1 if sem is locked
int sem_post(sem_t *s); // increment a semaphore
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Zeitmessung

6) Schreiben Sie ein Programm *ndaysago*, welches "heute, vor n Tagen" als Datum (gemäss Kalender) ausgibt, im Default-Format des Betriebssystems, wie hier im Beispiel. Punkte: __ / 10

```
$ date
Sun May 31 13:37:01 CEST 2026
$./ndaysago 3
Thu May 28 13:37:01 2026
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
int atoi(const char *nptr); // convert a string to an integer
int printf(const char *format, ...); // format string %s, char %c, int %d

char *asctime(const struct tm *tm); // return date string, default format
struct tm *localtime(const time_t *t); // get broken-down local time
time_t mktime(struct tm *tm); // convert broken-down local time to time_t
// ignores tm_wday, tm_yday; values outside valid interval are normalised
time_t time(time_t *t); // get local time in seconds since Epoch

struct tm {
    int tm_sec, tm_min, tm_hour; // (0-60), (0-59), (0-23)
    int tm_mday; // Day of the month (1-31)
    int tm_mon; // Month (0-11)
    int tm_year; // Year - 1900
    int tm_wday; // Day of the week (0-6, Sunday = 0)
    int tm_yday; // Day in the year (0-365, 1 Jan = 0)
    ... };
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

7) Schreibe ein Programm *count*, das eine mit 0 initialisierte *long* Variable eine Sekunde lang – in User CPU Zeit – inkrementiert, und dann den aktuellen Wert ausgibt. Punkte: _ / 12

```
$ time ./count
1870499
# output of time:
real    0m2.555s
user    0m1.000s
sys     0m1.555s
```

Verwende die folgenden Calls (soweit sinnvoll), ohne *#includes* und Fehlerbehandlung:

```
clock_t times(struct tms *t); // returns the number of clock ticks that
have elapsed since an arbitrary point in the past; stores the current
process times in the struct tms that t points to.
```

```
struct tms {
    clock_t tms_utime; // user time
    clock_t tms_stime; // system time
    clock_t tms_cutime; // user time of children
    clock_t tms_cstime; // system time of children
};
```

The number of clock ticks per second is:
sysconf(_SC_CLK_TCK)

```
int printf(const char *format, ...); // format string %s, char %c, int %d
```

Idee (kurz) und C Source Code hier, oder auf Zusatzblatt mit Namen und Frage-Nr.:

Zusatzblatt zu Aufgabe Nr. ____ von (Name) _____